

VerdictMMA

Documentacion Tecnica del Proyecto

Generado: 09/12/2025 16:13

Stack: Vue 3 + Vite + Supabase + Tailwind CSS

INDICE DE CONTENIDOS

1. Informacion General del Proyecto
2. Stack Tecnologico
3. Estructura de Directorios
4. Componentes (src/components/)
5. Paginas (src/pages/)
6. Servicios (src/services/)
7. Composables (src/composables/)
8. Router y Rutas
9. Estructura de Datos
10. Caracteristicas Principales

1. INFORMACION GENERAL DEL PROYECTO

Descripcion

VerdictMMA es una aplicacion web social enfocada en la comunidad de artes marciales mixtas (MMA). Permite a los usuarios crear publicaciones, interactuar mediante comentarios, chatear publicamente y de forma privada, y gestionar sus perfiles con un sistema de rangos.

Datos del Proyecto

- * Nombre: VerdictMMA
- * Tipo: Single Page Application (SPA)
- * Idioma: Espanol
- * Total de archivos en src: 35
- * Lineas de codigo aprox: 7,846
- * Tamano total src: 336 KB

2. STACK TECNOLÓGICO

Frontend

- * Vue.js 3.5.18 - Framework reactivo progresivo
- * Vue Router 4.5.1 - Sistema de enrutamiento SPA
- * Vite 7.1.2 - Build tool y dev server
- * Tailwind CSS 4.1.12 - Framework CSS utilities-first

Backend (BaaS)

- * Supabase 2.57.4 - Backend as a Service
- Autenticación de usuarios
- Base de datos PostgreSQL
- Storage para archivos
- Real-time subscriptions

3. ESTRUCTURA DE DIRECTORIOS

/VerdictNew/

public/ - Archivos estaticos publicos

src/ -Codigo fuente principal

main.js - Punto de entrada

App.vue - Componente raiz

style.css - Estilos globales (Tailwind)

components/ - 12 componentes reutilizables

pages/ - 9 vistas/paginas

services/ - 7 servicios de logica

composables/ - 3 composables Vue 3

router/ - Configuracion de rutas

index.html - HTML principal

package.json - Dependencias npm

vite.config.js - Configuracion Vite

4. COMPONENTES (src/components/)

darkNavbar.vue (24.3 KB)

Barra de navegacion principal con tema oscuro.

Funcionalidades:

- * Menu de navegacion responsivo
- * Busqueda de usuarios con autocompletado
- * Menu de usuario autenticado
- * Toggle de menu movil
- * Integracion con sistema de autenticacion

PublicationCard.vue (16.4 KB)

Card para mostrar publicaciones individuales.

Props: publication, currentUserId

Eventos: edit, delete

Funcionalidades:

- * Muestra titulo, contenido, categoria e imagen
- * Botones de editar/eliminar para el autor
- * Badge de categoria con icono
- * Link al perfil del autor
- * Formateo de fechas

EditPublication.vue (14.7 KB)

Modal para editar publicaciones existentes.

Props: show, publication

Eventos: close, updated

Funcionalidades:

- * Formulario de edicion completo
- * Cambio de imagen
- * Selector de categoria
- * Validacion de campos

CreatePublication.vue (13.2 KB)

Modal para crear nuevas publicaciones.

Props: show

Eventos: close, created

Funcionalidades:

- * Formulario de creacion
- * Subida de imagen opcional
- * Selector de 7 categorias
- * Validacion de titulo y contenido

CommentsList.vue (10.2 KB)

Lista de comentarios de una publicacion.

Props: publicationId

Funcionalidades:

- * Carga y muestra comentarios
- * Formulario para nuevo comentario
- * Paginacion/carga infinita
- * Edicion y eliminacion de propios

ProfileHeader.vue (6.2 KB)

Encabezado de perfil con informacion principal.

Props: profile

Funcionalidades:

- * Avatar grande del usuario
- * Nombre y rango
- * Badge PRO si aplica
- * Biografia

ProfileCard.vue (6.4 KB)

Card compacta de perfil para listas.

Props: profile

Funcionalidades:

- * Avatar pequeno
- * Nombre con link
- * Indicador de rango

CommentCard.vue

Card individual de comentario.

Props: comment, currentUserId

Eventos: edit, delete

Funcionalidades:

- * Muestra autor, fecha y contenido
- * Opciones de editar/eliminar

ToastNotification.vue (4.2 KB)

Sistema de notificaciones emergentes.

Funcionalidades:

- * Tipos: success, error, info, warning
- * Auto-dismiss configurable
- * Posicionamiento flexible
- * Animaciones de entrada/salida

RankBadge.vue (3.9 KB)

Badge que muestra el rango del usuario.

Props: rango, showProgress

Rangos: Novato, Aprendiz, Luchador, Guerrero, Veterano, Experto, Maestro, Leyenda, Campeon, Hall of Fame

AppH1.vue (168 B)

Componente simple de titulo H1 estilizado.

myFooter.vue (159 B)

Pie de pagina minimalista.

5. PAGINAS (src/pages/)

Home.vue (14.0 KB) - Ruta: /

Pagina de inicio/landing page.

No requiere autenticacion.

Funcionalidades:

- * Hero section con gradientes
- * Call-to-action para registro/login
- * Informacion sobre la plataforma
- * Secciones de características

Login.vue (7.9 KB) - Ruta: /login

Formulario de inicio de sesion.

No requiere autenticacion.

Funcionalidades:

- * Campo de email
- * Campo de contraseña
- * Validacion de formulario
- * Redireccion post-login
- * Link a registro

Register.vue (11.3 KB) - Ruta: /register

Formulario de registro de usuarios.

No requiere autenticacion.

Funcionalidades:

- * Campos: email, contraseña, nombre de usuario
- * Validacion de campos
- * Verificacion de contraseña
- * Creacion de perfil automatica

Publications.vue (14.8 KB) - Ruta: /publicaciones

Feed principal de publicaciones.

Requiere autenticacion.

Funcionalidades:

- * Lista de publicaciones paginadas
- * Filtro por categoria
- * Boton crear publicacion
- * Carga infinita o paginacion

Profile.vue (21.0 KB) - Ruta: /profile/:id?

Pagina de perfil de usuario.

Autenticacion opcional (publico).

Funcionalidades:

- * Muestra perfil propio o de otros
- * Lista de publicaciones del usuario
- * Estadisticas del perfil
- * Informacion de rango
- * Boton de chat privado (si no es propio)

Settings.vue (32.4 KB) - Ruta: /settings

Configuracion completa de cuenta.

Requiere autenticacion.

Funcionalidades:

- * Cambio de avatar
- * Edicion de nombre de usuario
- * Edicion de biografia
- * Cambio de contraseña
- * Vista previa de perfil
- * Informacion de rango

PublicChat.vue (16.8 KB) - Ruta: /chat

Chat publico en tiempo real.

No requiere autenticacion para ver.

Funcionalidades:

- * Mensajes en tiempo real
- * Lista de usuarios conectados
- * Envio de mensajes
- * Scroll automatico

PrivateChat.vue (10.8 KB) - Ruta: /chat/:displayName

Chat privado entre dos usuarios.

Requiere autenticacion.

Funcionalidades:

- * Conversacion 1 a 1
- * Historial de mensajes
- * Envio de mensajes
- * Marcado como leído

MessagesView.vue (6.3 KB) - Ruta: /messages

Vista de bandeja de mensajes.

Requiere autenticacion.

Funcionalidades:

- * Lista de conversaciones
- * Ultimo mensaje de cada conversacion
- * Indicador de no leidos
- * Busqueda de conversaciones

6. SERVICIOS (src/services/)

supabase.js

Inicializacion del cliente Supabase.

Exporta: supabase (cliente)

Configuracion de URL y clave API publica.

auth.js (3.2 KB)

Servicios de autenticacion.

Funciones exportadas:

- * register(email, password, displayName) - Registra nuevo usuario
- * login(email, password) - Inicia sesion
- * logout() - Cierra sesion
- * getCurrentUser() - Obtiene usuario actual
- * updateProfile(updates) - Actualiza metadata
- * subscribeToAuthChanges(callback) - Escucha cambios de auth

profiles.js (10.4 KB)

Gestion de perfiles de usuarios.

Constantes: RANGOS [Novato -> Hall of Fame]

Funciones exportadas:

- * getProfile(userId) - Obtiene perfil por ID
- * getProfileByIdentifier(identifier) - Busca por ID o slug
- * getCurrentProfile() - Perfil del usuario actual
- * updateProfile(updates) - Actualiza perfil propio
- * createProfile(userId, profileData) - Crea nuevo perfil
- * searchProfiles(searchTerm, limit) - Busqueda
- * getProfiles(page, pageSize) - Lista paginada
- * getProfileStats(userId) - Estadisticas
- * createSlugFromDisplayName(name) - Genera slug
- * getRangoIndex(rango) - Indice del rango
- * getRangoByIndex(index) - Rango por indice

publications.js (8.6 KB)

Gestion de publicaciones/posts.

Categorias: Articulo, Tutorial, Opinion, Review, Guia, Entrevista, Historia

Funciones exportadas:

- * createPublication(data) - Crea publicacion
- * getPublications(page, pageSize, category) - Lista con filtros
- * getPublication(id) - Una publicacion
- * getPublicationsByUser(userId) - Por usuario
- * updatePublication(id, updates) - Actualiza
- * deletePublication(id) - Elimina
- * subscribeToPublicationsChanges() - Real-time
- * getPublicationCategoryName(key) - Nombre categoria
- * getPublicationCategoryIcon(key) - Icono categoria

comments.js (6.2 KB)

Gestion de comentarios en publicaciones.

Funciones exportadas:

- * createComment(publicationId, content) - Crea comentario
- * getComments(publicationId, page, pageSize) - Lista
- * updateComment(commentId, content) - Actualiza
- * deleteComment(commentId) - Elimina
- * getCommentsCount(publicationId) - Cuenta total

private-chat.js (11.5 KB)

Servicios de mensajeria privada.

Funciones exportadas:

- * getConversations() - Lista conversaciones
- * getMessages(conversationId) - Mensajes de conversacion
- * sendMessage(recipientId, content) - Envia mensaje
- * markAsRead(conversationId) - Marca como leido
- * getUnreadCount() - Cuenta no leidos
- * findConversation(userId) - Busca conversacion
- * subscribeToMessages(callback) - Real-time

storage.js (10.9 KB)

Gestion de archivos en Supabase Storage.

Funciones exportadas:

- * uploadAvatar(file, userId) - Sube avatar
- * uploadPublicationImage(file, publicationId) - Sube imagen
- * deleteFile(bucket, path) - Elimina archivo
- * getPublicUrl(bucket, path) - URL publica
- * getAvatarUrl(userId) - URL de avatar

7. COMPOSABLES (src/composables/)

useAuth.js (2.5 KB)

Composable global para autenticacion reactiva.

Estado reactivo (refs):

- * user - Usuario autenticado actual
- * loading - Estado de carga
- * initialized - Flag de inicializacion

Propiedades computadas:

- * isAuthenticated - Boolean de autenticacion
- * userEmail - Email del usuario
- * userDisplayName - Nombre para mostrar
- * userId - ID unico del usuario

Metodos:

- * initialize() - Inicializa estado de auth
- * refreshUser() - Refresca datos del usuario
- * clearUser() - Limpia estado al logout

useProfile.js (5.7 KB)

Composable para gestion de perfiles.

useProfile() - Perfil del usuario actual:

- * Estado: currentProfile, profileLoading
- * Datos: displayName, bio, avatarUrl, rango, isPro, profileId
- * Info rango: rangoIndex, rangoProgress
- * loadCurrentProfile() - Carga perfil actual
- * updateCurrentProfile(updates) - Actualiza
- * clearCurrentProfile() - Limpia

useExternalProfile() - Perfiles de otros:

- * Cache de perfiles cargados
- * loadProfile(userId) - Carga perfil externo
- * getCachedProfile(userId) - Obtiene del cache

useToast.js (1.9 KB)

Composable para sistema de notificaciones.

Estado:

- * visible - Boolean de visibilidad
- * message - Mensaje a mostrar
- * type - Tipo: success, error, info, warning

Metodos:

- * showToast(message, type, duration) - Muestra toast
- * hideToast() - Oculta toast
- * showSuccess(message) - Toast de exito
- * showError(message) - Toast de error

8. ROUTER Y RUTAS (src/router/)

router.js (3.4 KB)

Configuración de Vue Router 4.

Rutas públicas:

- * / -> Home.vue (Página de inicio)
- * /login -> Login.vue (Inicio de sesión)
- * /register -> Register.vue (Registro)
- * /chat -> PublicChat.vue (Chat público)
- * /profile/:id? -> Profile.vue (Perfil público)

Rutas protegidas (requieren autenticación):

- * /publicaciones -> Publications.vue (Feed)
- * /settings -> Settings.vue (Configuración)
- * /chat/:displayName -> PrivateChat.vue (Chat privado)
- * /messages -> MessagesView.vue (Bandeja)

Navigation Guards:

- * beforeEach - Verifica meta.requiresAuth
- * Redirige a /login si no autenticado
- * Redirige a perfil propio si /profile sin ID

9. ESTRUCTURA DE DATOS

User (Supabase Auth)

```
{
  id: UUID,
  email: string,
  user_metadata: {
    display_name: string
  }
}
```

Profile

```
{
  id: UUID (same as auth user id),
  display_name: string,
  bio: string | null,
  avatar_url: string | null,
  rango: string (Novato...Hall of Fame),
  pro: boolean,
  created_at: timestamp
}
```

Publication

```
{
  id: UUID,
  user_id: UUID,
  title: string,
  content: string,
  category: string,
  image_url: string | null,
  created_at: timestamp,
  updated_at: timestamp
}
```

Comment

```
{
  id: UUID,
  publication_id: UUID,
  user_id: UUID,
  content: string,
  created_at: timestamp,
  updated_at: timestamp
}
```

Message (Chat Privado)

```
{
  id: UUID,
  sender_id: UUID,
```



```
recipient_id: UUID,  
content: string,  
read: boolean,  
created_at: timestamp  
}
```

10. CARACTERISTICAS PRINCIPALES

Autenticacion

- * Registro con email/contrasena
- * Login/logout
- * Sesion persistente
- * Guards de rutas protegidas

Sistema de Perfiles

- * 10 niveles de rango: Novato -> Hall of Fame
- * Avatar personalizado
- * Biografia editable
- * Estado PRO
- * Busqueda de usuarios

Publicaciones

- * CRUD completo
- * 7 categorias de contenido
- * Soporte de imagenes
- * Paginacion y filtros

Comentarios

- * Sistema completo de comentarios
- * Edicion y eliminacion
- * Control de permisos

Mensajeria

- * Chat publico en tiempo real
- * Chat privado 1 a 1
- * Historial persistente
- * Indicadores de no leidos

UI/UX

- * Tema oscuro por defecto
- * Diseno responsivo
- * Notificaciones toast
- * Tailwind CSS utilities